Lab Report 2

IEEE 754 Conversion & HMMM Assembly Programming

Student Name: [Helin Özmen]

Student ID: [230543028]

Object-Oriented Program Design
March 5, 2026

Contents

1 Part 1 – IEEE 754 Pen & Paper Exercises **2**
1.1 Exercise 1 – Positive Integer: 58 2
1.2 Exercise 2 – Positive Fractional: 5.75 2
1.3 Exercise 3 – Negative Integer: -44 2
1.4 Exercise 4 – Negative Fractional: -7.25 2
1.5 Exercise 5 – Reverse Conversion 2

2 Part 2 – HMMM Assembly Programs **3**
2.1 Task 1 – Area of a Square 3
2.2 Task 2 – Positive or Negative? 3
2.3 Task 3 – Factorial 3

3 Reflection **4**

1 Part 1 – IEEE 754 Pen & Paper Exercises

1.1 Exercise 1 – Positive Integer: 58

- **Binary Conversion:** $58 = 111010_2$
- **Normalization:** 1.11010×2^5 (Exponent is 5)
- **Exponent Calculation:** $127 + 5 = 132 \rightarrow 10000100_2$
- **Sign Bit:** 0
- **Final Pattern:** `0 10000100 11010000000000000000000`

1.2 Exercise 2 – Positive Fractional: 5.75

- **Binary Conversion:** $5.75 = 101.11_2$
- **Normalization:** 1.0111×2^2 (Exponent is 2)
- **Exponent Calculation:** $127 + 2 = 129 \rightarrow 10000001_2$
- **Sign Bit:** 0
- **Final Pattern:** `0 10000001 01110000000000000000000`

1.3 Exercise 3 – Negative Integer: -44

- **Binary Conversion:** $44 = 101100_2$
- **Normalization:** 1.01100×2^5 (Exponent is 5)
- **Exponent Calculation:** $127 + 5 = 132 \rightarrow 10000100_2$
- **Sign Bit:** 1
- **Final Pattern:** `1 10000100 01100000000000000000000`

1.4 Exercise 4 – Negative Fractional: -7.25

- **Binary Conversion:** $7.25 = 111.01_2$
- **Normalization:** 1.1101×2^2 (Exponent is 2)
- **Exponent Calculation:** $127 + 2 = 129 \rightarrow 10000001_2$
- **Sign Bit:** 1
- **Final Pattern:** `1 10000001 11010000000000000000000`

1.5 Exercise 5 – Reverse Conversion

- **Given Pattern:** `0 10000101 10100000000000000000000`
- **Sign Bit:** 0 (Positive)
- **Exponent:** $10000101_2 = 133$. Real exponent: $133 - 127 = 6$
- **Mantissa:** 1.101_2 (including hidden bit)
- **Calculation:** $1.101_2 \times 2^6 = 1101000_2$
- **Decimal Result:** $64 + 32 + 8 = 104$

2 Part 2 – HMMM Assembly Programs

2.1 Task 1 – Area of a Square

Pseudocode

Read the side length s, calculate the square of s, and write the area result.

HMMM Code

```
0  read  r1          # Input side length 's'
1  mul   r2  r1  r1  # r2 = s * s
2  write r2          # Output area
3  halt              # End
```

Simulator Output

2.2 Task 2 – Positive or Negative?

Pseudocode

Read input. If positive, write 1. If zero, write 0. If negative, write -1.

HMMM Code

```
0  read  r1          # Read n
1  jgtzn r1  5       # Jump to POSITIVE if n > 0
2  jltzn r1  7       # Jump to NEGATIVE if n < 0
3  write r1          # Else, write 0
4  halt              # End
5  setn  r2  1       # POSITIVE logic
6  write r2
7  halt
8  setn  r3  -1      # NEGATIVE logic
9  write r3
10 halt
```

2.3 Task 3 – Factorial

Pseudocode

Set result to 1. Multiply result by N, decrement N until it reaches zero.

HMMM Code

```
0  read  r1          # Input N
1  setn  r2  1       # result = 1
2  jeqzn r1  6       # Exit loop if N == 0
3  mul   r2  r2  r1  # result = result * N
4  addn  r1  -1      # N = N - 1
5  jump  2           # Repeat loop
6  write r2          # Output result
7  halt
```

Brief Explanation

The program uses a loop that iterates until the input register $r1$ (representing N) reaches zero. In each iteration, it multiplies the current result in $r2$ by N and then decrements N by 1.

3 Reflection

During this laboratory, I learned the detailed process of IEEE 754 floating-point conversion and low-level assembly programming logic. Working with HMMM helped me understand how conditional jumps and loops are implemented at the machine level.